Implementação de um Serviço SOAP de Calculadora

Desenvolvimento de Aplicações Web 2026/27 – 1.º Trabalho Prático

Marcelo Santos
a79433@ualg.pt

Outubro de 2026

Contents

1 Objetivo e âmbito **2**

2 Enquadramento e requisitos **2**

3 Tecnologias e organização **2**

3.1 Porquê Bun 2

3.2 Separação entre frontend e backend 2

4 Contrato WSDL e operações **3**

5 Implementação do servidor e validação **3**

6 Exemplos do protocolo **4**

7 Clientes e demonstração no SoapUI **6**

8 Testes e resultados observados **11**

9 Instalação, execução e entrega **11**

10 Conclusão e limitações **11**

1 Objetivo e âmbito

Este relatório documenta a implementação e o consumo de um serviço SOAP sobre HTTP para as quatro operações aritméticas básicas. O serviço usa a biblioteca `soap` e as APIs HTTP de Node.js, com execução através de Bun. Os clientes disponíveis são uma aplicação web e um projeto SoapUI.

2 Enquadramento e requisitos

O enunciado solicita um servidor SOAP com adição, subtração, multiplicação e divisão, um WSDL que identifique entradas e saídas, validação numérica e tratamento da divisão por zero. Exige ainda um cliente capaz de visualizar requisições e respostas HTTP/SOAP, testes e documentação de execução. Os critérios de avaliação distribuem 100 pontos entre operações (20), estrutura (10), WSDL (20), validação (10), clientes (20), documentação (10) e boas práticas (10).

A solução implementada cobre as quatro operações e oferece duas formas de as consumir: uma aplicação React no navegador e o SoapUI com quatro requisições configuradas. Os testes automatizados verificam o serviço e a aplicação. Há, porém, uma diferença face à exigência literal de um cliente simples em Node.js: não existe um programa CLI Node.js separado. A demonstração programática existente é o cliente TypeScript no navegador, e os testes são executados com Bun. Esta distinção é relevante na avaliação.

3 Tecnologias e organização

O serviço está em TypeScript e usa `node:http` para criar o servidor HTTP, `node:fs` para ler o contrato e a biblioteca npm `soap` (versão 1.14.0) para publicar as operações. O ambiente de execução usado é Bun 1.4.4, compatível com as APIs de Node.js utilizadas; portanto, não se afirma que o processo esteja a correr no binário `node`. A interface usa React e `fetch` para criar e enviar XML SOAP 1.1 diretamente a partir do navegador.

3.1 Porquê Bun

Optei por Bun pela rapidez procurada na instalação de dependências e na execução das tarefas de desenvolvimento, em comparação com o fluxo habitual de `npm` mais ferramentas separadas. Não foi realizado um benchmark neste projeto, pelo que a escolha expressa uma preferência de desenvolvimento, não uma medição de desempenho deste serviço. Bun reúne num só ambiente o gestor de pacotes (`bun install`), a execução de TypeScript (`bun src/index.ts`), o executor de testes (`bun test`) e a construção do frontend (`bun run build.ts`). `npm` é um gestor de pacotes, enquanto Bun também é um runtime; a comparação de velocidade refere-se sobretudo ao fluxo de instalação e ferramentas, não a uma comparação direta entre dois runtimes. As dependências continuam a ser pacotes do ecossistema npm, como `soap` e `react`.

3.2 Separação entre frontend e backend

Preferi manter o servidor SOAP e o cliente React em aplicações distintas para tornar visível a fronteira HTTP. O navegador, servido na porta 3001, consulta a configuração em `/api/config` e faz o `POST` diretamente ao serviço na porta 3000, sem proxy das chamadas SOAP. Assim, é possível observar o método, a URL e os cabeçalhos do pedido (`Content-Type` e `SOAPAction`) e

o estado, os cabeçalhos acessíveis e o XML da resposta, além de demonstrar a configuração de CORS exigida por origens diferentes.

O navegador envia o cabeçalho `Origin` e pode iniciar uma requisição preliminar `OPTIONS` (preflight). Para a origem autorizada, o backend responde com `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods` e `Access-Control-Allow-Headers`; este último inclui `Content-Type` e `SOAPAction`. O cabeçalho `Vary: Origin` distingue respostas de origens diferentes em caches. Esta separação permite explicar por que a política CORS é necessária para o navegador, enquanto o SoapUI, como cliente fora do browser, não depende dessa política.

Elemento	Responsabilidade
`soap-server/src/index.ts`	Implementação das operações, verificação de operandos e resultados, publicação SOAP e política CORS.
`soap-server/src/service.wsdl`	Contrato de mensagens, operações, binding SOAP e endereço do serviço.
`soap-app/src/service.ts`	Criação do envelope, envio HTTP, interpretação da resposta e dos faults.
`soap-app/src/APITester.tsx`	Formulário e visualização de resultado, detalhes HTTP e XML das duas direções.
`compose.yaml` e `Dockerfile`	Três serviços: backend, frontend e desktop remoto do SoapUI.

O fluxo de uma operação é: o utilizador escolhe a operação e os números; o cliente constrói um `Envelope` XML; envia um `POST` com `Content-Type: text/xml` e `SOAPAction`; a biblioteca `soap` encaminha a operação publicada no WSDL; a lógica valida, calcula e devolve `result`, ou gera um `SOAP Fault`. O cliente interpreta o `Body` pelo namespace SOAP e apresenta o XML e o estado HTTP.

4 Contrato WSDL e operações

O contrato é publicado em http://localhost:3000/wsdl?wsdl; as chamadas SOAP são dirigidas a http://localhost:3000/wsdl. O `targetNamespace` é http://example.org. O WSDL define mensagens de pedido com duas partes, `a` e `b`, e mensagens de resposta com uma parte `result`; todas são declaradas `xsd:double`. O `portType` contém quatro operações de pedido/resposta. O binding usa SOAP sobre HTTP, estilo `rpc` e `literal`, com `soapAction` igual ao nome da operação.

Operação	Argumentos	Cálculo	Exemplo
`Add`	`a, b`	$a + b$	$2 + 3 = 5$
`Subtract`	`a, b`	$a - b$	$9 - 4 = 5$
`Multiply`	`a, b`	$a \times b$	$6 \times 7 = 42$
`Divide`	`a, b`	a/b	$8/2 = 4$

Por exemplo, a operação `Add` dispõe das mensagens `AddRequest` e `AddResponse`; o mesmo padrão é repetido para as restantes operações. O WSDL é lido no arranque, e o endereço publicado é atualizado de acordo com `SOAP_PORT`. A tipagem `xsd:double` descreve o contrato numérico; a validação adicional de valores finitos é feita na implementação do serviço.

5 Implementação do servidor e validação

A função `calculate` centraliza a verificação de entrada e saída, evitando regras diferentes nas quatro operações. Primeiro recusa operandos inexistentes, nulos ou constituídos apenas por

espaços. Depois converte os valores com `Number` e rejeita aqueles que não são finitos, como `NaN` e `Infinity`. Por fim, verifica que o resultado também é finito. A divisão adiciona uma regra explícita: quando `b=0`, gera `faultstring` igual a `divisao por zero`.

Os erros de entrada são expostos como `SOAP Fault` com `faultcode Client` e estado HTTP 500. Por exemplo, `Add(oops, 2)` é rejeitado como entrada não numérica, e `Divide(1, 0)` não devolve `Infinity`. Em vez disso, a resposta contém uma falha SOAP legível pelo cliente. Uma política CORS autoriza a origem da interface web e responde a preflight `OPTIONS`, necessário porque o navegador envia `SOAPAction` e XML por `POST`.

6 Exemplos do protocolo

O exemplo seguinte representa uma chamada SOAP 1.1 para `Add(2,3)`. O servidor recebe um pedido HTTP `POST /wsdl` com `Content-Type: text/xml; charset=utf-8` e `SOAPAction: Add`. Prefixos de namespace como `tns` e `exam` são equivalentes quando indicam o mesmo URI.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:tns="http://example.org">
<soap:Body>
<tns:Add><a>2</a><b>3</b></tns:Add>
</soap:Body>
</soap:Envelope>
```

Uma resposta bem-sucedida tem HTTP 200 e a parte `result` dentro de `AddResponse`:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:tns="http://example.org">
<soap:Body>
<tns:AddResponse><tns:result>5</tns:result></tns:AddResponse>
</soap:Body>
</soap:Envelope>
```

Para `Divide(1,0)`, a resposta real apresenta HTTP 500 e o seguinte conteúdo lógico dentro de `soap:Body`:

```
<soap:Fault>
<faultcode>Client</faultcode>
<faultstring>divisao por zero</faultstring>
</soap:Fault>
```

As capturas da Figura 1 e da Figura 2 mostram os estados HTTP e os documentos SOAP expostos lado a lado pelo cliente web. Os cabeçalhos apresentados pela interface são os disponibilizados ao JavaScript do navegador; a inspeção integral do transporte pode ser feita nas ferramentas de rede do navegador ou no SoapUI.

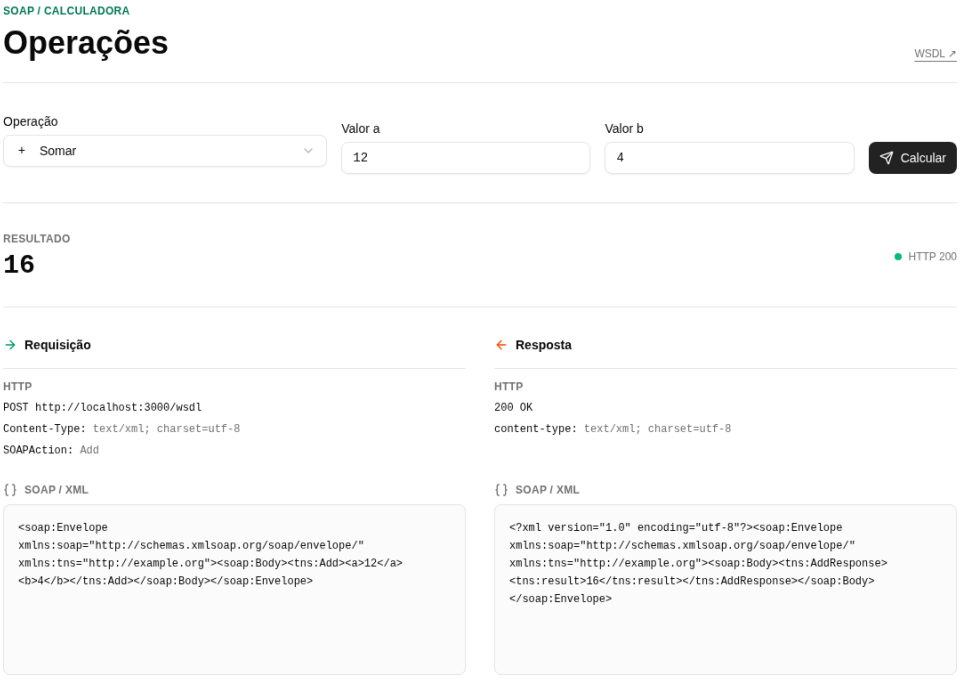


Figure 1: Cliente web: requisição Add(12,4), estado HTTP 200 e resultado 16 no XML da resposta.

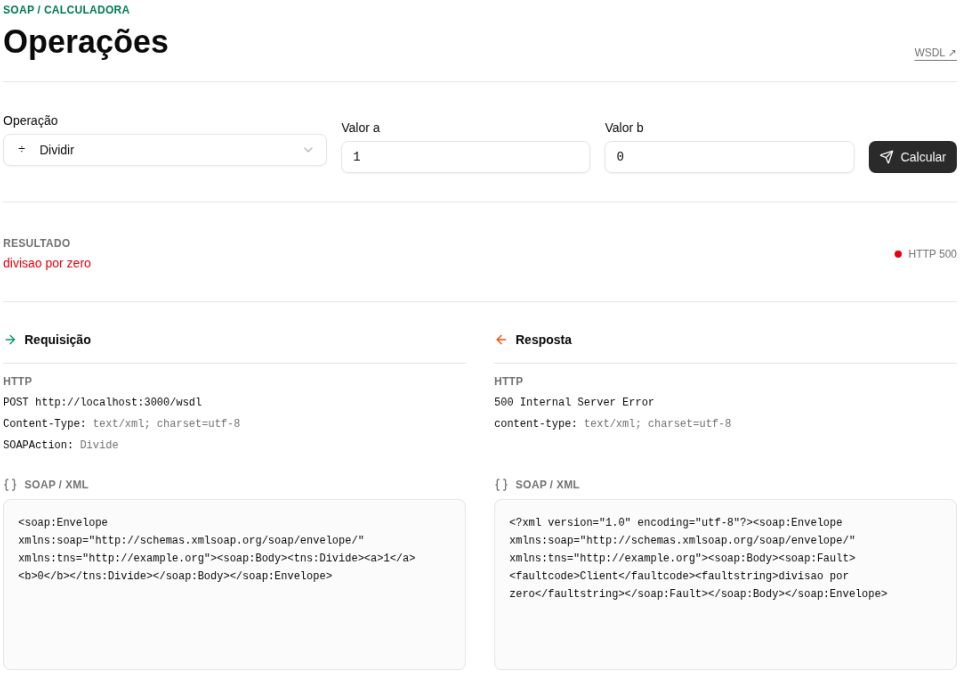


Figure 2: Cliente web: Divide(1,0), estado HTTP 500 e SOAP Fault com a mensagem de divisão por zero.

7 Clientes e demonstração no SoapUI

No cliente web, o formulário recebe a operação e dois números. Antes da chamada, recusa entradas vazias e não finitas; `service.ts` obtém a URL de `/api/config`, monta o envelope SOAP e envia-o por `fetch`. O XML da resposta é analisado com `DOMParser`; o cliente procura um `Fault` no `Body` ou um elemento `OperationResponse` com `result`. São conservados o pedido, a resposta, o resultado e eventuais mensagens de erro, para que o ecrã mostre o tráfego e não apenas o número calculado.

O frontend usa React 19 para gerir o estado do formulário, do resultado e da troca HTTP/SOAP. Os controlos visuais seguem o padrão *shadcn/ui*: o ficheiro `components.json` configura o estilo e os componentes `Button`, `Input`, `Label` e `Select` estão no código da aplicação, em `soap-app/src/components/ui/`. Shadcn/ui não é, neste caso, uma biblioteca de componentes importada como dependência única em tempo de execução; é uma coleção de componentes copiáveis e personalizáveis, construída com Tailwind CSS e, em alguns controlos, primitivas Radix UI. Os ícones são fornecidos por `lucide-react`. A lógica SOAP permanece separada destes componentes de interface.

O SoapUI 5.9.1 está acessível como aplicação gráfica num desktop XFCE remoto em https://localhost:8080/ (certificado local autoassinado). O projeto `Calculadora SOAP` foi importado a partir de http://soap-server:3000/wsdl?wsdl e guardado no volume persistente `soapui-config`, no ficheiro `/config/Calculadora-SOAP-soapui-project.xml`. Contém uma requisição de exemplo por operação, com valores válidos e endpoint http://soap-server:3000/wsdl.

Esta diferença de endereços é deliberada: `localhost:3000` é apropriado para o navegador no host; no contêiner do SoapUI, `localhost` apontaria para o próprio desktop remoto. Por isso, o SoapUI usa o nome de serviço `soap-server` da rede Docker Compose. O WSDL anuncia `localhost` para o consumidor no host; a definição importada no SoapUI e as respetivas requisições usam a URL interna. O projeto guardado mantém estas escolhas entre reinícios, desde que o volume não seja removido.

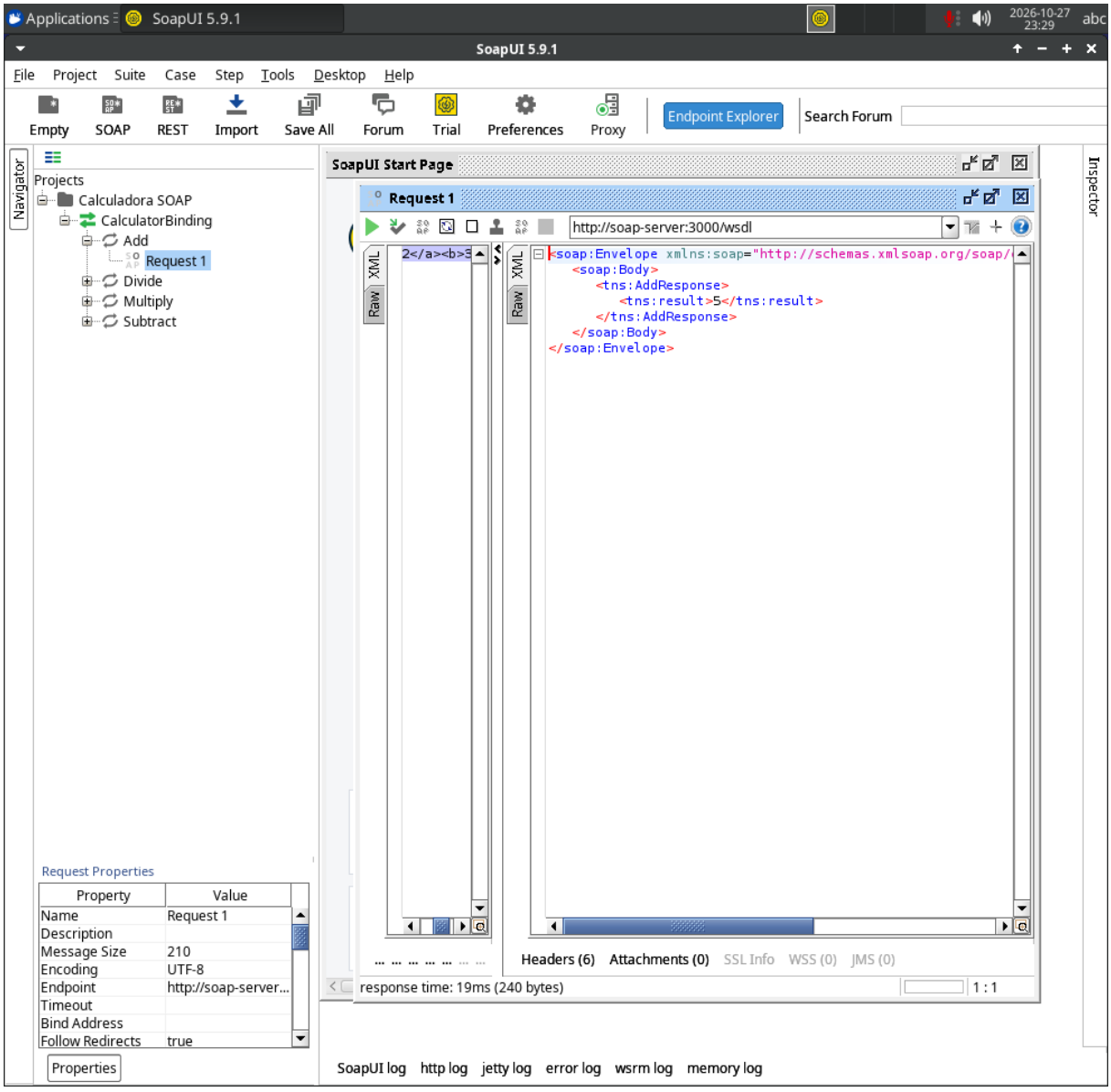


Figure 3: SoapUI com a operação Add(2,3): resposta SOAP recebida com resultado 5.

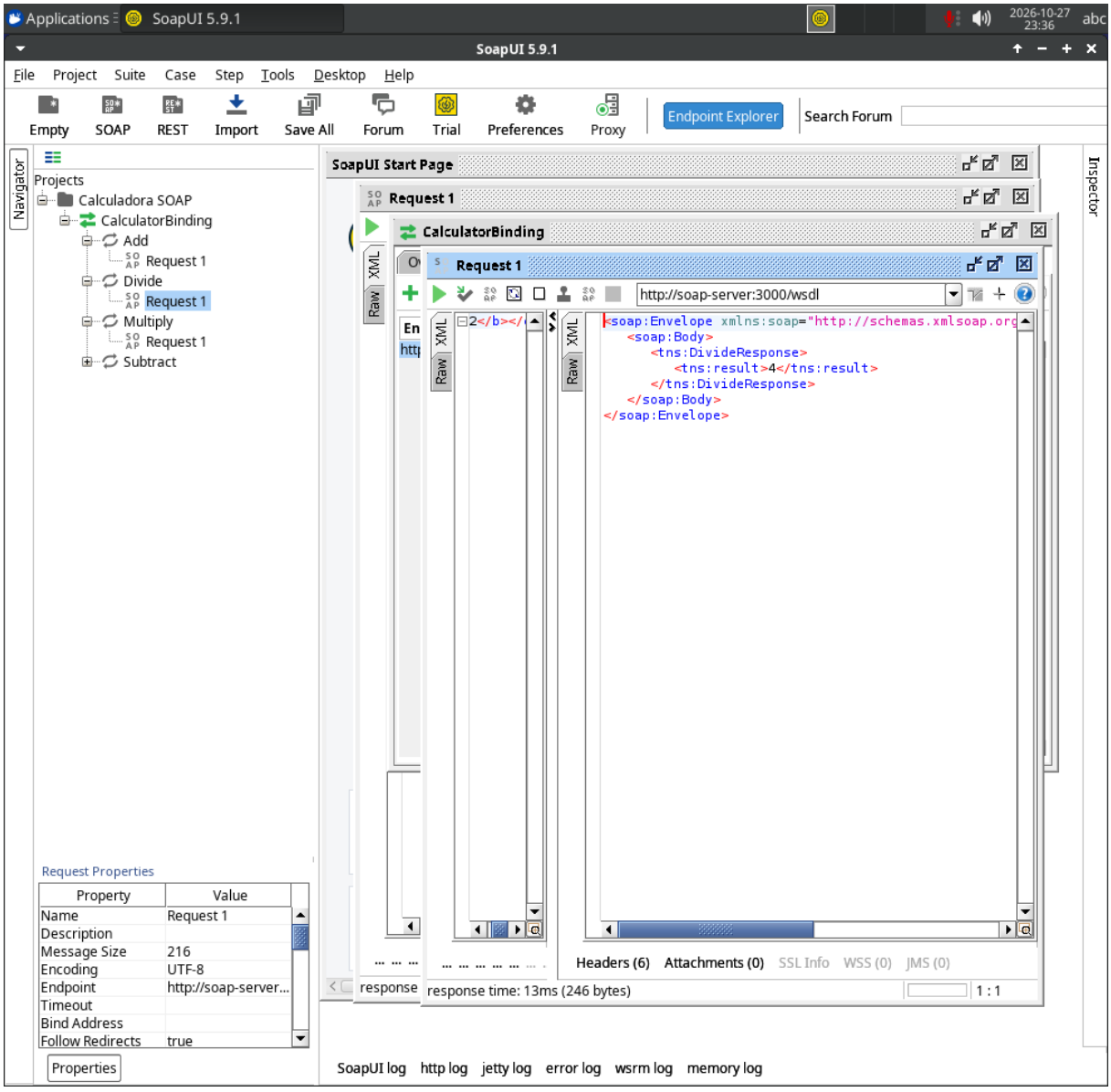


Figure 4: SoapUI com a operação Divide(8,2): resposta SOAP com resultado 4.

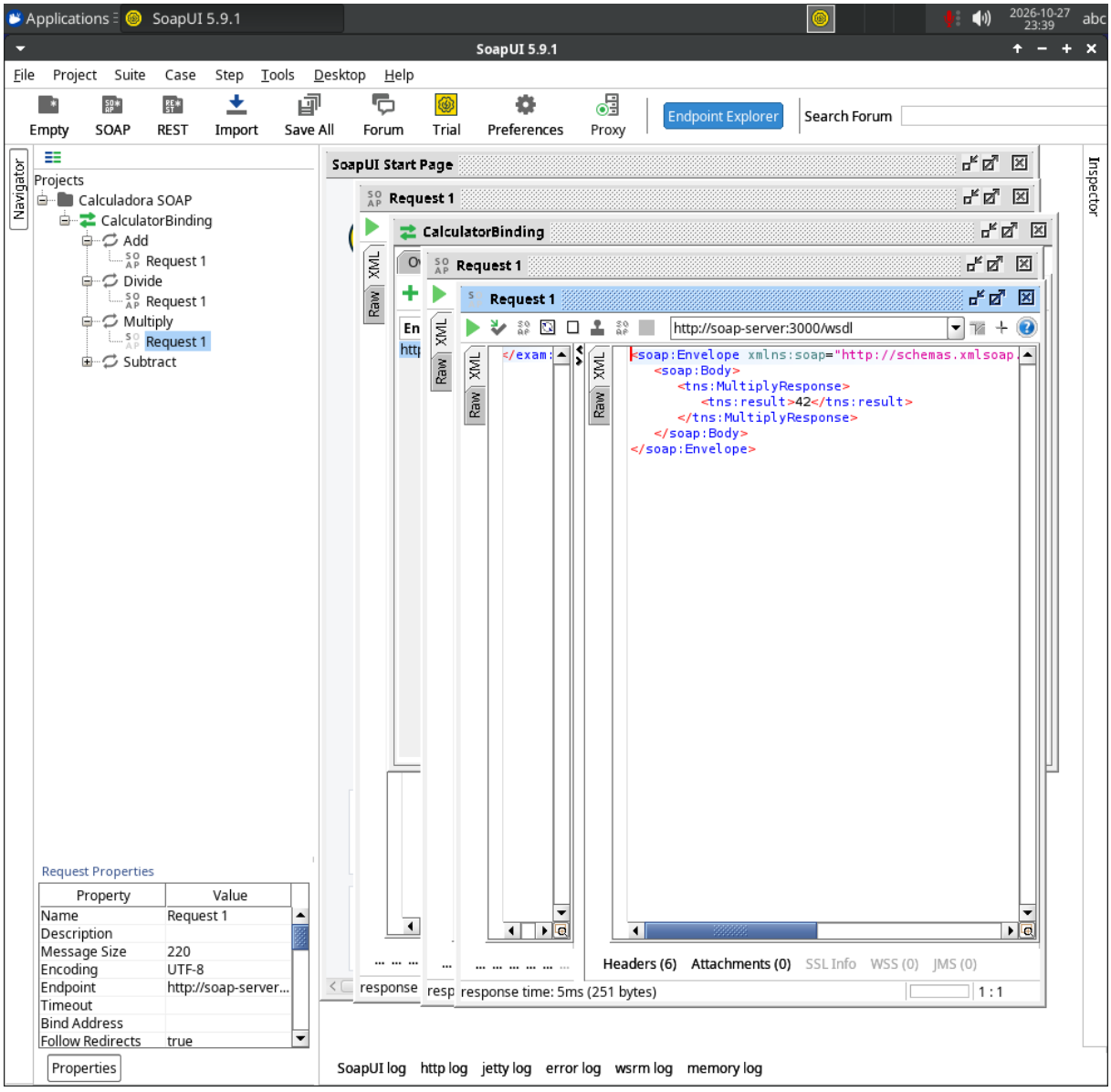


Figure 5: SoapUI com a operação Multiply(6,7): resposta SOAP com resultado 42.

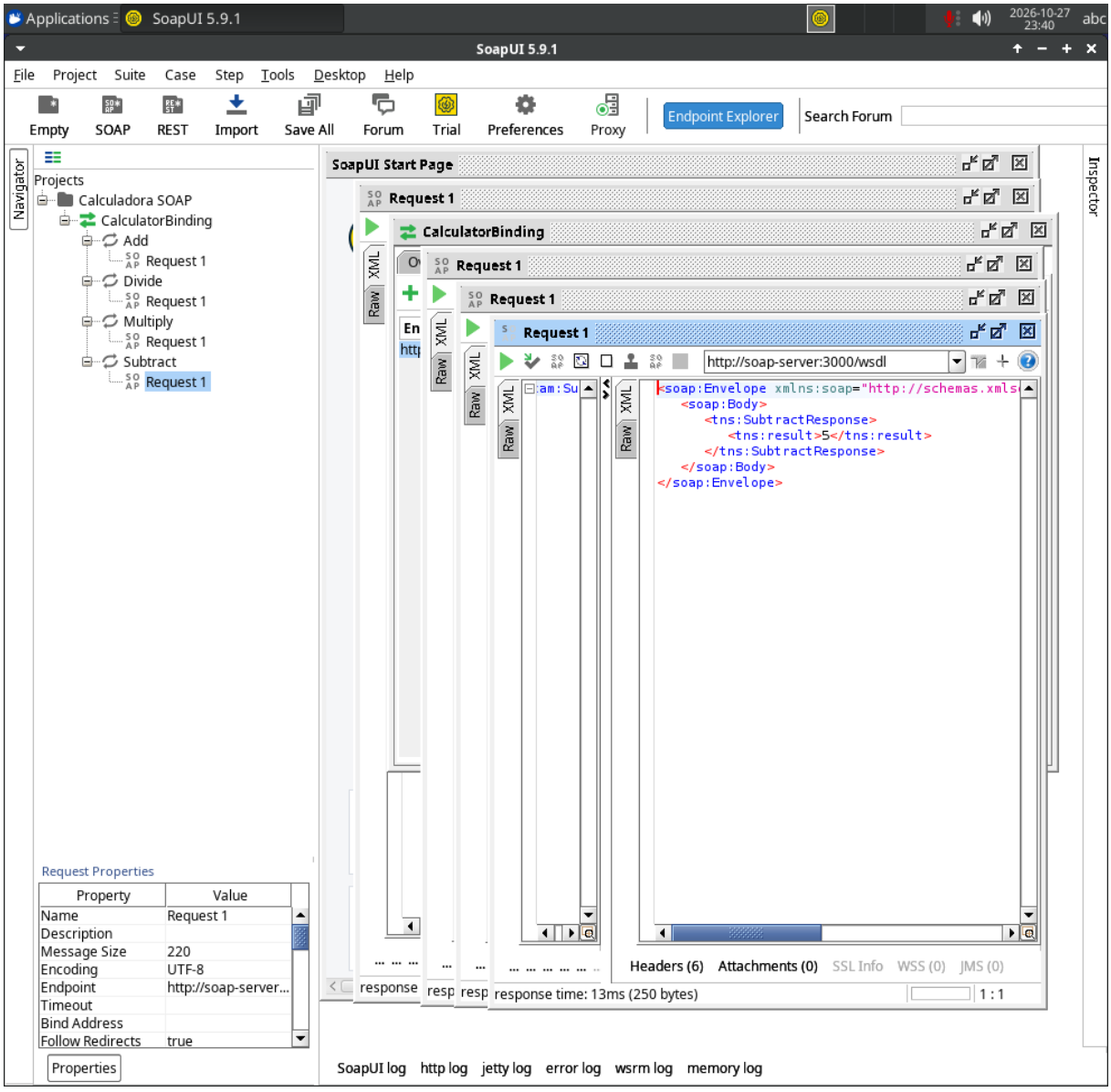


Figure 6: SoapUI com a operação Subtract(9,4): resposta SOAP com resultado 5.

8 Testes e resultados observados

Foram executadas as suites `bun test` de ambos os pacotes: **12 testes passaram no servidor** e **6 testes passaram na aplicação**, sem falhas. Os testes do servidor verificam a publicação do WSDL, as quatro operações, entradas inválidas (incluindo valores vazios, `NaN` e `Infinity`), operandos ausentes, divisão por zero e CORS. Os testes do cliente usam Chromium para verificar as respostas e os detalhes HTTP/XML, faults, XML malformado, indisponibilidade do serviço, validação local e ausência de deslocamento horizontal no ecrã móvel.

Cenário	Entrada	HTTP / saída	Evidência
Soma	2, 3	200 / 5	SoapUI, Figura 3
Subtração	9, 4	200 / 5	SoapUI, Figura 6
Multiplicação	6, 7	200 / 42	SoapUI, Figura 5
Divisão	8, 2	200 / 4	SoapUI, Figura 4
Divisão por zero	1, 0	500 / SOAP Fault	Cliente web, Figura 2; testes
Argumento inválido	`oops`, 2	500 / SOAP Fault	Teste automatizado do servidor

As quatro requisições de exemplo do projeto SoapUI foram inspecionadas após guardar: as quatro usam o endpoint interno e contêm os operandos indicados na tabela. As capturas são evidência visual das chamadas manuais; os resultados das suites referem-se à execução automatizada descrita acima e não substituem a inspeção do protocolo.

9 Instalação, execução e entrega

Sem Docker, é necessário instalar Bun. Na pasta `soap-server`, executar `bun install` e `bun src/index.ts`; noutra consola, na pasta `soap-app`, executar `bun install` e `bun dev`. Abrir `http://localhost:3001/` e consultar o contrato em `http://localhost:3000/wsdl?wsdl`. Para executar os testes, usar `bun test` em cada pasta.

Com Docker Compose, executar `docker compose up -build` na raiz do repositório. O backend fica na porta 3000, a aplicação na 3001 e o SoapUI em `https://localhost:8080/`. A porta do SoapUI está limitada a `127.0.0.1`. Aceitar o certificado autoassinado ao abrir o desktop; selecionar uma das quatro requisições existentes no projeto `Calculadora SOAP` e premir o botão de envio. Não usar `docker compose down -v` se se pretender conservar o projeto no volume.

O código do serviço está em `soap-server/`, o cliente em `soap-app/`, o WSDL em `soap-server/src/service.wsdl` e as instruções principais em `README.md`. O ficheiro de projeto SoapUI encontra-se no volume Docker, e não é um ficheiro versionado do repositório. As figuras deste relatório foram capturadas a partir da aplicação em execução e do SoapUI configurado.

10 Conclusão e limitações

O serviço publica um contrato para as quatro operações, processa os pedidos SOAP e devolve respostas numéricas ou faults para entradas inadequadas. O cliente web e o SoapUI permitem demonstrar o consumo do serviço e examinar as mensagens trocadas. A separação entre contrato, lógica de negócio, cliente e interface torna mais direta a verificação de cada requisito.

Como limitações relevantes, a execução usa Bun em vez do executável Node.js pedido literalmente, não foi criado um cliente de linha de comando Node.js independente e o projeto SoapUI depende do volume Docker para persistir. O endereço publicado no WSDL é `localhost`, pelo

que clientes dentro de contêineres precisam de configurar o endereço interno. Estas condições devem ser explicitadas durante a apresentação do trabalho.